Data Structures & Systems Design — Take-Home Assignment

Avaloka (Guruvai Sciences) — AI / Multi-Agent Systems Intern
Name: Ganesh Barmavath
Date: 21 October 2026
Language: Python 3

Problem 1: LRU Cache

My approach

In an LRU cache, we keep only a fixed number of items. When the cache is full and a new item comes in, the item that was not used for the longest time has to be removed. So the cache has to do two things well: find any item quickly, and always know which item is the oldest one.

I used two data structures together, because one alone was not enough.

- **Hash map (dictionary):** it gives me the item for a key directly. I do not have to search through all the items, so this part is fast, O(1).
- **Doubly linked list:** I keep all the items in a list, in the order they were used. The front of the list is the item used most recently, and the last item in the list is the oldest one. When an item is used, I remove it from its place and put it at the front. In a linked list, this only means changing a few links (pointers), so this is also O(1). And when I have to remove the oldest item, I always know where it is — it is the last one in the list.

The dictionary stores the key and also points to the item's place (node) in the list. That is the important part. Because of this, I don't search in the list at all. The dictionary takes me straight to the node, and then I only change the links around that node.

One more point: even a `get` call counts as a use. So in my code, `get` also moves the item to the front, not only `put`.

I also used two dummy nodes, one at the start (`head`) and one at the end (`tail`) of the list. They do not store any data. They only make the code simple — because of them, I don't need separate checks for an empty list or for the first and last item.

Before this, I thought of using a normal list or array and moving the used item to the front every time. But in an array, all the other items have to shift one place, which takes O(n) time. So I did not use that idea.

Code

```
class _Node:
    """One entry in the doubly linked list."""
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}                 # key -> _Node
        # Sentinel (dummy) head and tail so insertions/removals never
        # need special cases for empty ends.
        # head.next  = most recently used,  tail.prev = least recently used
        self.head = _Node()
        self.tail = _Node()
        self.head.next = self.tail
        self.tail.prev = self.head
    # -- internal helpers (all O(1) pointer work) ---------------------------
```

```
    def _remove(self, node: _Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_to_front(self, node: _Node) -> None:
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def _move_to_front(self, node: _Node) -> None:
        self._remove(node)
        self._add_to_front(node)

    # -- public API ---------------------------------------------------------
    def get(self, key: int) -> int:
        node = self.cache.get(key)
        if node is None:
            return -1
        self._move_to_front(node)       # a read also counts as "used"
        return node.value

    def put(self, key: int, value: int) -> None:
        node = self.cache.get(key)
        if node is not None:            # update existing key
            node.value = value
            self._move_to_front(node)
            return

        node = _Node(key, value)
        self.cache[key] = node
        self._add_to_front(node)

        if len(self.cache) > self.capacity:   # evict least recently used
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]
```

Example to understand (capacity is 2):

`put(1, 1)` and `put(2, 2)` — cache now has 1 and 2. `get(1)` gives 1, and now 1 becomes the most recent item. `put(3, 3)` — cache is full, so the oldest item, 2, is removed. `get(2)` now gives -1. Then `put(4, 4)` removes 1. `get(3)` gives 3 and `get(4)` gives 4. If I `put` a key that is already in the cache, I only change its value and make it the most recent item. Nothing is removed in that case.

Problem 2: Event Scheduler

My approach

First function — can one person attend all the events?

I first sort all the events by their start time. After sorting, I only have to check each event with the one just before it. If an event starts before the previous event has ended, there is an overlap, and my answer is False. If no event overlaps with the previous one, then the person can attend all the events, so the answer is True.

If one event ends at 10 and the next event starts at 10, that is not an overlap. The person can attend both. So in my code, I check this carefully — overlap happens only when the start time is *less than* the previous end time.

Second function — how many rooms are needed?

The number of rooms needed is equal to the maximum number of events that are going on at the same time. If, at any time, 3 events are running together, I need 3 rooms. My code finds this number in a simple way.

I go through the events one by one, in the order of their start time. I keep the end times of the rooms that are busy in a **min-heap**. A min-heap always keeps the smallest end time at the top, so I can quickly see

which room will become free first.

1. For every new event, I check only one thing: has the room that ends earliest already finished before this event starts?
2. If yes, I give that same room to this event and update its end time in the heap.
3. If no, it means all the rooms are still busy, so I have to take one more room.

In the end, the total number of rooms I opened is my answer. I don't have to check all the rooms for every event, only the room that ends first. That is why the heap is useful here.

Code

```
def can_attend_all(events) -> bool:
    """True if no two events overlap. Adjacent events (end == start) are OK."""
    if len(events) <= 1:
        return True
    ordered = sorted(events, key=lambda e: e[0])
    for i in range(1, len(ordered)):
        if ordered[i][0] < ordered[i - 1][1]:   # starts before previous ends
            return False
    return True


def min_rooms_required(events) -> int:
    """Minimum rooms so every event can run; reuses a room as soon as it frees."""
    if not events:
        return 0
    ordered = sorted(events, key=lambda e: e[0])
    end_times = []                          # min-heap of end times of rooms in use
    heapq.heappush(end_times, ordered[0][1])
    rooms = 1
    for start, end in ordered[1:]:
        if end_times[0] <= start:       # earliest-ending room is free again
            heapq.heapreplace(end_times, end)
        else:                           # all rooms busy -> need a new one
            heapq.heappush(end_times, end)
            rooms = max(rooms, len(end_times))
    return rooms
```

Example: for `[(9, 10), (9, 11), (10, 12)]`, the first two events start at 9, so both are going on at the same time — I need 2 rooms. The third event starts at 10, and the first room becomes free at 10, so it uses the first room again. The answer is 2 rooms, and `can_attend_all` is False. For `[(9, 10), (10, 11), (11, 12)]`, no events overlap, so 1 room is enough and `can_attend_all` is True.

Final Discussion & Analysis

1. Time and space complexity

- `LRUCache.get` **— Time: O(1), Extra space: O(1).** I search the key in the dictionary and change a few links to bring the node to the front. These are small, fixed steps, so the time does not grow when the cache becomes big.
- `LRUCache.put` **— Time: O(1), Extra space: O(1).** It is also the same. I search the dictionary, change a few links, and if the cache is full, I remove the last node, which also takes only a few steps.
- **LRU Cache total memory — Space: O(capacity).** For every item in the cache, I store one node in the list and one entry in the dictionary.
- `can_attend_all` **— Time: O(n log n), Extra space: O(n).** Most of the time goes in sorting the events by start time. After sorting, I go through the list only one time and check each event with the previous one, and that part is O(n). I make a sorted copy of the events, so the space is O(n).
- `min_rooms_required` **— Time: O(n log n), Extra space: O(n).** Sorting takes O(n log n) time. For every event, I do one work in the heap — I either change the top end time or add one new end time — and one heap work takes O(log n) time. The heap keeps one end time for every room that is in use, and the rooms can go up to n, so the space is O(n).

- `assign_rooms` **(Question 3) — Time: O(n log n), Extra space: O(n).** It does the same steps as `min_rooms_required`, with only one more small heap for the room numbers that have become free.

Here O(1) means average time. A dictionary is very fast in normal cases, but in a rare case where many keys get the same hash, it can become slow. This is the same for almost every solution that uses a hash map.

2. Trade-offs — why use a hash map and a doubly linked list together?

If I use **only a hash map**, I can find any item fast. But the hash map does not keep the order of use, so when the cache is full, I cannot find the oldest item quickly. I would have to check all the items, which is O(n).

If I use **only a linked list**, the order of use is clear. The oldest item is at the end and the newest item is at the front. But to find one item, I have to go through the list one by one, which is again O(n).

So I used both together. The hash map helps me reach the item fast, and the linked list helps me change its position and remove the oldest item fast. Together, both `get` and `put` become O(1).

The list has to be a **doubly** linked list, not a single one. When I remove a node from the middle, I also have to change the link of the node before it. In a doubly linked list, every node knows the node before it, so this is easy. In a singly linked list, I would first have to search for that previous node.

There are also some disadvantages, and I should mention them. First, this uses more memory, because for every item I store one dictionary entry and one list node with two links. Second, the code is a little risky — if I miss changing even one link, the order of the list will become wrong, and it may not show as an error immediately. That is why I kept all the link-changing work inside three small functions, so there is only one place to check. I know Python has `OrderedDict`, which does the same thing from inside. In a real project, I would use that. Here, I wrote it myself so that I can show how the O(1) time is actually achieved.

3. How will I give a room number to each event?

This change is not very big, because my `min_rooms_required` function already decides, for every event, whether it will use an old room or a new room. I only have to remember that decision. I will do it like this:

1. In the heap, instead of keeping only the end time, I will keep the end time and the room number together, like `(end_time, room_number)`.
2. I will keep one more small heap for the rooms that have become free. When a new event comes, I first move all the rooms that have already ended into this free list.
3. Then I give the event the smallest free room number. If no room is free, I give it a new room number.
4. Events are checked in sorted order, but the final answer should be in the same order in which the events were given to me. So I also remember the original position of each event, and I write the room number in that position.

My code for this is below. It returns a list like `((9, 10), 1)`, which means the event from 9 to 10 got Room 1. In the app, Room 1, Room 2 and so on can easily be shown as Room A, Room B. The time and space do not change much — it is still O(n log n) time and O(n) space.

```
def assign_rooms(events):
    """Extension discussed in the analysis: give each event a room number.

    Returns a list of (event, room_number) in the ORIGINAL input order.
    Room numbers start at 1. A freed room number is reused before a new
    room is opened.
    """
    indexed = sorted(enumerate(events), key=lambda p: p[1][0])
    busy = []        # min-heap of (end_time, room_number)
    free = []        # min-heap of room numbers that have become available
    next_room = 0
    result = [None] * len(events)
    for idx, (start, end) in indexed:
```

```
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
        else:
            next_room += 1
            room = next_room
        heapq.heappush(busy, (end, room))
        result[idx] = ((start, end), room)
    return result
```

Example: `assign_rooms([(9, 10), (9, 11), (10, 12)])` gives `[((9, 10), 1), ((9, 11), 2), ((10, 12), 1)]`. The third event gets Room 1 again, because that room became free at 10.

4. How will I make the LRU Cache safe for many threads?

Threads means many users, or many parts of a program, using the same cache at the same time. My cache is not safe for that right now, and there is one reason which is easy to miss: in an LRU cache, even `get` changes something. It moves the node to the front of the list. So `get` is not a read-only operation here.

If two threads change the links of the list at the same time, the list can get spoiled — one item can get lost, or the links can point to the wrong nodes. In the same way, if two threads add new items together, both can see that there is space in the cache, and in the end the cache can have more items than its capacity. The dictionary and the list can also stop matching each other.

The simplest correct solution is to use **one lock**. In Python, this is `threading.Lock`. I will keep the full `get` and the full `put` inside the lock, so that at one time, only one thread is working on the cache. Locking only the dictionary will not be enough, because the problem is in the list also. The full work — searching, moving the node, and removing the old item — has to happen together, without any other thread coming in between.

Some people think Python's GIL will handle this. It will not. The GIL protects very small steps, but one `get` call has many steps, and another thread can start in the middle of it.

The disadvantage of one lock is speed. All the threads have to wait for their turn, so if there are too many threads, the cache can become slow. If that really happens, the next step is to divide the cache into parts, for example 16 small caches, each with its own lock. Threads using different parts will not have to wait for each other. The order of items will not be perfectly exact across all the parts, but for most real uses, that small difference does not matter much. I will not do this from the start. First, I will use one lock, because it is simple and correct, and I will change it only if I see a real speed problem.

One more care I will take: I will not call any outside code or function while the lock is held. If that code waits or tries to take the same lock again, the program can get stuck. This is how deadlocks usually start.

How I tested my code

For the LRU cache, I ran the example I have written above (capacity 2) and checked each answer one by one. I also checked that `get` gives -1 for a key that is not in the cache, and that updating a key does not remove any other item.

For the scheduler, I did not test only with my own examples. I wrote a small brute-force program that counts, in a simple and slow way, how many events are running at the same time. Then I compared my answers with its answers on 500 random sets of events, including empty lists and lists with only one event. I also did the same check for the room number function in Question 3. All the answers matched.